Automata

From state machines to agentic AI

Roberto Masocco
roberto.masocco@uniroma2.it

Tor Vergata University of Rome
Department of Civil Engineering and Computer Science Engineering

June 4, 2026

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

Roadmap

1. Finite-state machines
2. Behavior trees
3. Future directions

Roadmap

Automata in robotic systems

Beyond the lower layers of perception, planning, and control, every modern **autonomous system** needs an **orchestration layer**: the top of the stack where everything has become a **software abstraction**, **user input** can be parsed, and the **mission** is **executed**.

We present **three families** of possible **implementations** for this layer.

- **Finite-state machines**: the traditional starting point.
- **Behavior trees**: when FSMs become unwieldy.
- **LLM-based agents**: a glimpse of where the field is heading.

We use the term **automata** for this layer.
This is unrelated to the formal-language automata seen in other courses.

An FSM is a **mathematical model of computation**.

Formal definition

A finite-state machine is a tuple $(S, \Sigma, \delta, s_0, F)$, where

- S is a finite set of **states**;
- Σ is a finite set of **input symbols** (events, triggers);
- $\delta : S \times \Sigma \to S$ is the **transition function**;
- $s_0 \in S$ is the **initial state**;
- $F \subseteq S$ is the set of **terminal** (or *accepting*) **states**.

At any time, the machine is in **exactly one state** from S; an incoming event **triggers a transition** according to δ.

In practice, δ is encoded as a **table**: each **row** associates a **source state**, a **trigger**, and a **destination state**.

This is the form FSMs take in code: a **list** of triples, easy to read and easy to extend.

δ must be **well-defined**: every (state, trigger) pair must have at most one outgoing transition, otherwise the machine is non-deterministic and a different formalism is needed.

Finite-state machines

Diagrammatic representation

FSMs are usually drawn as **directed graphs**: circles are states, arrows are transitions labeled with the triggering event. An incoming arrow with no source marks the **initial state**; a double circle marks a **terminal state**.

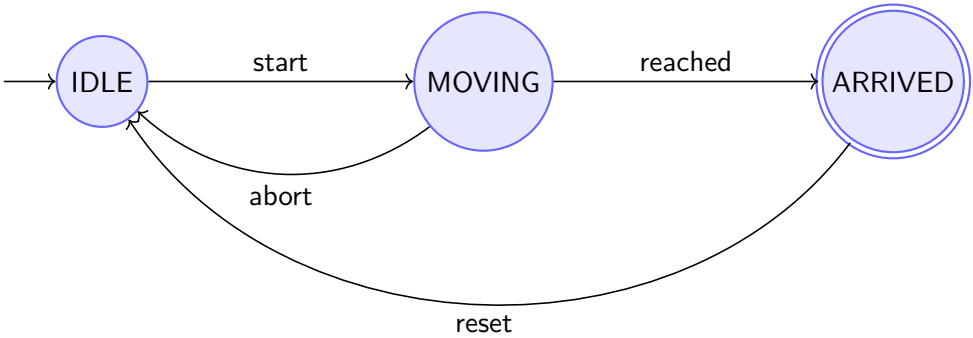


Figure 1: A simple 3-state FSM controlling a motion task.

A **passive** FSM is a **data structure**: it **tracks the state of something** (a component, a process, a robot), and is updated by external code.

The FSM itself does not run anything. It answers questions like the following.

- **Where am I?** (Current state)
- **Can I do X?** (Is the current state allowed to transition on trigger X?)

Passive FSMs are observers and reporters, not executors.

Passive and active FSMs

Active

An **active** FSM **embeds execution logic**: a routine is associated to each state, and is **executed when the state is entered**.

The routine's **return value** may be the **trigger** for the next transition.
The machine **progresses autonomously** until a **terminal state** is reached.

Active FSMs are full executors of a mission or process.

Each step of an **active FSM** follows a **fixed cycle**:

1. the machine is in a **state** s with **associated routine** R_s;
2. R_s is invoked and **runs to completion**;
3. R_s **returns a trigger** $t \in \Sigma$;
4. the machine looks up $\delta(s, t) = s'$ and **transitions** to s';
5. if $s' \in F$ (*i.e.*, it is **terminal**), the **machine halts**; otherwise the cycle **repeats**.

An FSM run is a sequence of routine calls, threaded together by the transition table.

This execution model has direct consequences:

- **every routine must terminate**, otherwise the machine is stuck;
- **progress is strictly sequential**: one state, one routine, at a time;
- there is **no preemption**: while R_s runs, no external event can interrupt it.

Consequence

A routine that must react to multiple conditions has to monitor them explicitly inside its body: there is no FSM-level mechanism for reacting to events occurring during state execution.

transitions_ros

Our reference implementation is the transitions_ros Python library, built on top of the transitions Python package.

Where transitions focuses on FSMs as **state trackers** attached to arbitrary objects, transitions_ros focuses on the FSM as a **controller**, and adds

- **full ROS 2 integration**: the machine owns a ROS node and routines can use it to communicate;
- explicit support for the **active** execution model described before;
- a **generic executor node** that **loads an FSM definition at runtime**.

The library is built around **two ideas**:

- a `Machine` class extending `transitions.Machine`, that optionally **holds a ROS 2 node** and **supports active routines**;
- a **generic ROS 2 node** (`machine_executor`) that **loads an FSM definition from parameters and a Python module**, then **runs it**.

The executor pattern

The `machine_executor` is **mission-agnostic**: it knows nothing about the specific FSM it runs. The FSM definition lives in a **separate Python module loaded at machine instantiation**. **Reconfiguring the mission means swapping the module**, not rebuilding the executor.

```
transitions_table = [
    ['START',          'INIT',           'WAIT_START'],
    ['READY',          'WAIT_START',     'FIRST_TAKEOFF'],
    ['AIRBORNE',       'FIRST_TAKEOFF',  'EXPLORE'],
    # ...
    ['MISSION_DONE',  'DONE',           'RTH'],
    ['AT_HOME',        'RTH',            'COMPLETED']
]
```

Listing 1: Example of transitions table in `transitions_ros`.

Each **row** encodes one entry of δ as `[trigger, source, destination]`.
Trigger strings are what state routines return when they want to **drive the corresponding transition**.

```python
def init_routine(node: MachineExecutorNode,
                 wait_servers: bool,
                 timeout_sec: float) -> str:
    """Wait for the mission start command."""
    node.get_logger().warn("SYSTEMS ONLINE")
    # ... do something, possibly use the ROS node ...
    return 'START'

states_table = [
    {'name': 'INIT',      'routine': init_routine},
    {'name': 'RTH',       'routine': rth_routine},
    {'name': 'COMPLETED', 'routine': completed_routine}
]
```

Listing 2: A state routine and a states table.

Once started, the **executor node**:

- **iterates the active machine** via `run()`, calling each routine in turn;
- **exposes the ROS node to the routines**, which can publish, subscribe, call services and actions, and use parameters as any ROS 2 client would;
- **can be stopped, reconfigured, and restarted** at runtime without rebuilding.

The mission is just Python code that uses ROS and returns the right trigger strings.

Such an orchestrator node is usually a client for ROS 2 servers running on the robot.

Use case: Leonardo Drone Contest

Overview of LDC21

- Indoor 20x10 m arena with static obstacles.
- 10 ArUco-marked pads are placed in known locations.
- Drone starts from a given pad.
- Drone searches a UGV on the floor with a given marker, and a sequence of numbers.
- Drone reports sequence of scores for each pad to GCS.
- Operators decide a landing sequence, drone executes.

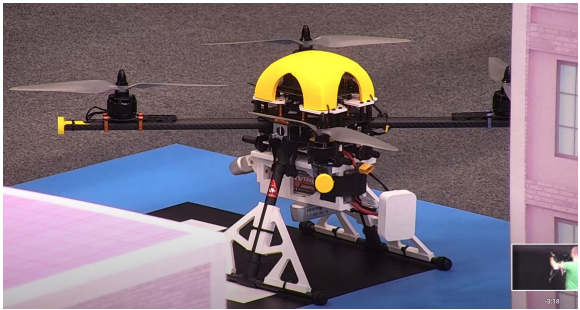


Figure 2: René drone on the starting pad (2021).

Use case: Leonardo Drone Contest

Issues of LDC21

Mission appears linear and **without reactive behaviors**, but

- **some steps require listening for events**, *e.g.*, a target is found;
- **must account for every abnormal condition**, *e.g.*,
  - landing pad not found;
  - landing pad unreachable;
- **user interaction is required** to get the landing sequence.

A pure software module is required, orchestrating execution by calling other modules.

Figure 3: “Io voglio la roba tua, la roba tua, ’n tanto al chilo, hai capito? La roba tua de ’na volta, voglio che apri tutto. Voglio che smarmelli!” (R. Ferretti)

Use case: Leonardo Drone Contest

Architecture design of LDC21

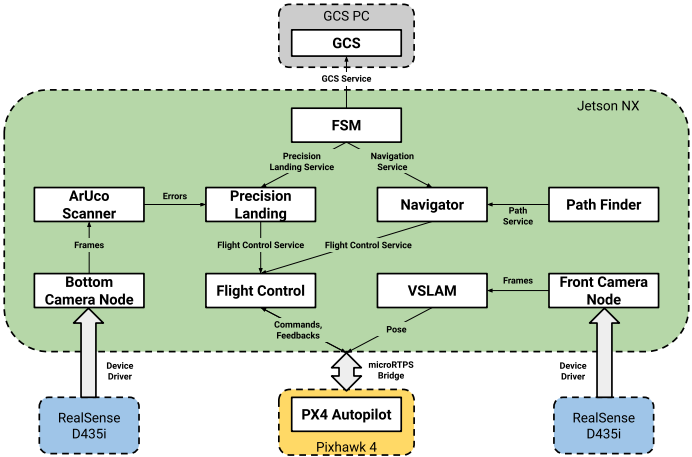


Figure 4: **Complete hierarchical software architecture scheme (2021).**

Use case: Leonardo Drone Contest

FSM diagram of LDC21

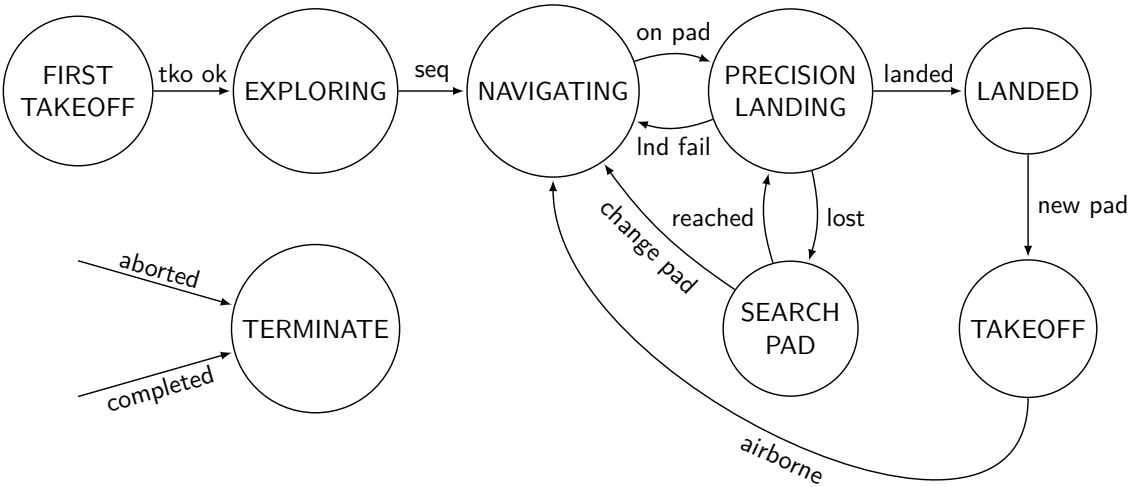


Figure 5: **"Dai dai dai!" (2021).**

Use case: Leonardo Drone Contest

Pain points of LDC23-LDC24

Mission was still linear(-ish) in LDC22, **things started to get wild from 2023 onwards**:

- **two agents**, a UAV and UGV, sharing a common network;
- **Human-Machine Interfaces (HMI)** as a requirement;
- **many more tasks**, some shared between UAV and UGV;
- **many more asynchronous events** with external triggers;
- **external software to interface** for mission generation and management.

We started to hit the limits of an FSM.

When FSMs stop scaling

The qualitative story

As soon as the mission becomes **reactive** — the robot must continuously **monitor multiple conditions** — **FSMs start to show their limitations**:

- **every monitored condition must be checked inside every routine**, else the event is missed;
- **adding a new condition often means touching many transitions** at once;
- **recovery and error-handling paths quickly outnumber the nominal ones**.

The FSM stops being a clean model of the mission and becomes a **tangle of spaghetti code**.

When FSMs stop scaling

The quantitative story

If the robot must independently track N binary conditions, the number of distinct **combinations of conditions** is 2^N.

In the worst case, faithfully modeling the mission requires **one state per combination**: the state space grows **exponentially**.

State explosion

$N = 5$ **conditions** $\rightarrow$ **up to** 32 **states.**
$N = 10$ **conditions** $\rightarrow$ **up to** 1024 **states.**

Even with disciplined modeling, the resulting machine is hard to read, hard to test, and hard to evolve.

When FSMs stop scaling

Hierarchical FSMs and statecharts

The classical mitigation is the **hierarchical FSM**, or **Harel statechart**: **states may contain nested sub-states**, with parallel regions and history points.

This is what one finds in **industrial tools** (UML state machines, MATLAB Stateflow) and it does tame explosion, but **the underlying model is still state-centric**:

- **reactivity must still be encoded as transitions**;
- **modularity is limited to nesting**;
- **composition of independent behaviors remains limited** by structural requirements.

A different idiom is needed.

Roadmap

1 Finite-state machines

2 Behavior trees

3 Future directions

From states to trees

We now turn to a different formalism, **born in videogame AI** and now **ubiquitous in robotics**: the **behavior tree** (BT).

Behavior trees are

- **naturally active**: an execution model is built into the formalism;
- **modular**: each subtree is a complete behavior, composable with others;
- **reactive by construction**: the tree is re-evaluated at every step.

The **reference implementation** is the C++ library `BehaviorTree.CPP`.

A behavior tree is a **directed rooted tree** whose **nodes encode the logic** of a mission.

Two families of nodes exist:

- **internal nodes** (**control nodes**, **decorators**) compose the behavior of their children;
- **leaf nodes** (**actions**, **conditions**) interface with the rest of the system.

The **tree** is the **program**; **ticking** it is the **execution**.

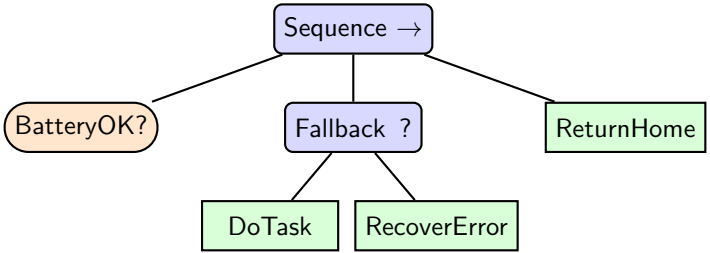


Figure 6: A small behavior tree, with a `Sequence` root, a nested `Fallback` subtree, an action (□), a condition (⬭) and a recovery alternative.

Execution is driven by **ticks**: a **periodic signal** sent from the **root** of the tree.

When ticked, an internal node **forwards the tick** to its children according to its own policy.

The traversal is always top to bottom, left to right.

The **order of children** therefore matters: it encodes **priority**. A tree is **single-threaded at the tick level** — the root sees a sequence of ticks, never a parallel one.

Every node, when ticked, **returns** one of three statuses:

- `SUCCESS`: the node has completed its task successfully;
- `FAILURE`: the node has failed;
- `RUNNING`: the node is in progress, and needs more ticks.

The three-status calculus

The **return value flows up** the tree from the ticked node to the root: each control node interprets the statuses of its children and computes its own status from them.

Leaves are where the tree meets the rest of the system.

- **Action nodes**: do actual work — move the robot, call a service, run a computation. They **may be asynchronous**, and can return `RUNNING` until completion.
- **Condition nodes**: evaluate a predicate — is the battery full? is the target visible? They are **synchronous** and can only return `SUCCESS` or `FAILURE`.

Conditions never block; actions may.

Control nodes

Overview

BT.CPP provides three main families of **nodes that control the execution flow**:

- **Sequence**: run children one after the other, all must succeed (AND);
- **Fallback** (or **Selector**): try children in order, at least one must succeed (OR);
- **Parallel**: tick all children in the same cycle.

Each family has **plain** and **reactive** variants, with subtly different semantics.

Control nodes

Sequence and Fallback

Plain **Sequence** ($\rightarrow$):

- ticks children left-to-right;
- returns `FAILURE` on the first `FAILURE`, `RUNNING` if a child is `RUNNING`, `SUCCESS` if all children succeed.

Plain **Fallback** (?):

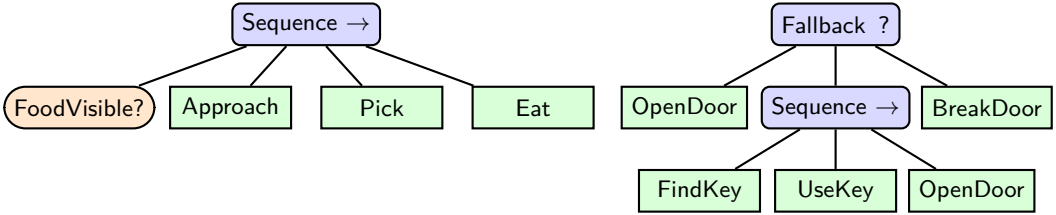
- ticks children left-to-right;
- returns `SUCCESS` on the first `SUCCESS`, `RUNNING` if a child is `RUNNING`, `FAILURE` if all children fail.

Memory of plain controls

On a re-tick after `RUNNING`, **plain controls resume from the running child**, skipping the children that have already completed.

Control nodes

Two examples



(a) Food gathering: nominal path.

(b) Door opening: alternatives.

Figure 7: Two canonical patterns: `Sequence`-based happy path (a) and `Fallback`-based recovery cascade (b).

ReactiveSequence and **ReactiveFallback** differ from their plain siblings on one point: on a re-tick, they do not skip the already-completed children.
They **re-tick every child from the first one, every tick**.

This is essential when **preconditions can change during execution**: a condition that succeeded one tick ago may have flipped, and a reactive control will catch it; a plain one will not.

Plain vs reactive

Plain controls have **memory** of past successes: they are appropriate when "done is done".
Reactive controls re-evaluate everything every tick: they are appropriate when the world keeps changing.

Control nodes

Parallel and ParallelAll

The **Parallel** family ticks **all children in the same cycle**, then **aggregates** their statuses.

Parallel uses **thresholds**:

- returns `SUCCESS` as soon as a **configurable number** of children **succeed**;
- returns `FAILURE` as soon as a **configurable number** of children **fail**.

ParallelAll is stricter: it **waits for every child to complete** before deciding. No early exit.

Both are the right tool when **several independent behaviors must progress together** — for example, navigate while keeping an eye on the battery and on a target detector.

Control nodes

Concurrency, not parallelism

Parallel does **not** mean "parallel threads". The tree is single-threaded: within one tick cycle, a `Parallel` node ticks its children **one after another, in order**.

What enables apparent simultaneity is the fact that **action leaves can be asynchronous**: an action returns `RUNNING` immediately, while the underlying computation **keeps progressing between ticks**.

The right mental model

Concurrent progress of long-running actions, **sequential traversal** of the tree.
Long would-be-blocking work belongs in an asynchronous action, never inside the tick of a synchronous one (unless specific exceptions, see later).

Decorators

A **decorator** is an internal node with **exactly one child**: it modifies the child's behavior in some way.

The most common decorators in BT.CPP:

- **Inverter**: swaps `SUCCESS` and `FAILURE`;
- **Retry**: re-ticks the child up to N times on `FAILURE`;
- **Repeat**: re-ticks the child up to N times on `SUCCESS`;
- **Timeout**: fails the child if it does not finish within a time budget;
- **ForceSuccess**, **ForceFailure**: ignore the child's status and return a fixed one.

Decorators turn primitives into reusable, parameterized behaviors.

The blackboard

Nodes need to **exchange data**: the result of a detection action must reach the navigation action, a parameter must reach the right leaf, and so on.

`BT.CPP` solves this with the **blackboard**: a **typed, shared key-value store accessible to all nodes** in the tree.

Nodes declare **ports**:

- **input ports**: read from the blackboard;
- **output ports**: write to the blackboard.

Ports can be **wired** as in, *e.g.*, a Simulink diagram, to implement a flow of information across ticks.

BT.CPP loads trees from **XML files**: the structure of the tree is data, separate from the implementation of its nodes. A **scripting language** can be used to perform operations on variables in the XML.

```
<root BTCPP_format="4">
  <BehaviorTree ID="GatherFood">
    <Sequence name="root">
      <FoodVisible target="{food_pose}"/>
      <Approach    goal="{food_pose}"/>
      <Pick        target="{food_pose}"/>
      <Eat/>
    </Sequence>
  </BehaviorTree>
</root>
```

Listing 3: Example XML file for the food-gathering tree; the first three leaf nodes have ports; they are all wired together pointing the same blackboard entry `food_pose` (curly braces).

Each node is a C++ class inheriting from a base provided by the library.

There are **four relevant categories**:

- **actions**;
- **conditions**;
- **controls**;
- **decorators**.

Each node is a C++ class inheriting from a base provided by the library.

Actions may be of multiple **types**.

`SyncActionNode` has only a `tick()` method, which must return either `SUCCESS` or `FAILURE`.

`StatefulActionNode` may return `RUNNING` for prolonged operations and be **interrupted**.

- `onStart()` method must gather data and **start the operation**, returning `RUNNING` if all is good;
- `onRunning()` method **does the job** and may return any of the three states;
- `onHalted()` is invoked **when the operation must stop** to clean up, as in **preemption**.

`CoroActionNode` may be used to implement actions that can be **paused**.

Each node is a C++ class inheriting from a base provided by the library.

Conditions inherit from `ConditionNode`.

They are **leaves that may only do a quick check** in their `tick()` method, returning either `SUCCESS` or `FAILURE`.

Almost everything can be done with a `ScriptCondition` node.

Each node is a C++ class inheriting from a base provided by the library.

Controls inherit from `ControlNode`. They

- may have **multiple children**;
- implement a **policy** to decide if, when, and how many children get ticked afterwards.

The library offers many Controls which implement many **common control flow constructs**, it is unlikely that you may need to add more.

Each node is a C++ class inheriting from a base provided by the library.

Decorators inherit from `DecoratorNode`. They

- have **only a single child**;
- may **alter** the result of their child;
- may **control when or how many times** their child is ticked;
- may tick their child imposing **conditions or constraints**.

The library offers many Decorators which implement many **unary operators**, it is unlikely that you may need to add more.

dua_behaviortree_cpp

Behavior Trees meet ROS 2

The dua_behaviortree_cpp repository integrates BT.CPP with ROS 2.

It houses **four packages**:

- btcpp_executor: **composable node** acting as the **BT executor**;
- dua_btcpp_base: **base class** for custom BT.CPP node extensions, built on pluginlib;
- dua_btcpp_types: **custom data types** for the integration;
- dua_btcpp_nodes: a **library of ready-made nodes** for common use cases (everyday actions, services, etc.).

You may **implement your node library** by **inheriting from the register class** in dua_btcpp_base.

The dua_behaviortree_cpp framework is **designed for continously-running BTs**.

Thus, in dua_btcpp_nodes and btcpp_executor especially, the following **convention on node return states** is adopted.

- RUNNING is returned when all is well, and propagated to the root to signal that **the tree is still being executed**.
- SUCCESS shall be returned by instantaneous checks and conditions or by actions, but **only propagated to the root when the tree has to stop successfully**.
- FAILURE shall be returned **on any unrecoverable error** and **immediately propagated to the root**.

At startup, the `btcpp_executor`:

1. **reads its parameters**: tree XML path, list of BT node **plugin libraries** to load;
2. **loads each plugin library** through `pluginlib`, registering every node it exposes into the BT.CPP factory;
3. parses the XML and **instantiates the tree**;
4. starts **ticking**.

Composable node

Being a **composable node**, the executor can be loaded into the same process as other ROS 2 nodes, sharing intra-process communications and reducing latency for the action and service calls issued by the BT leaves.

The executor pattern

transitions_ros vs dua_behaviortree_cpp

The same architectural idea appears in both formalisms.

transitions_ros

A generic **machine_executor** node loads, at runtime, a Python module containing:

- a transitions table;
- a states table with per-state routines.

The mission lives in the module, not in the executor.

dua_behaviortree_cpp

A generic **btcpp_executor** node loads, at runtime:

- a tree XML;
- one or more node plugin libraries.

The mission lives in the XML and the plugins, not in the executor.

Same pattern, two formalisms: generic executor, runtime-loaded domain logic.

Use case: Leonardo Drone Contest

Overview of LDC25

- **Localize** ArUco, QR, COCO objects.
- Execute **tasks** within **deadlines**.
  - **FindTarget** (with pic).
  - **FollowSequence** (with pic).
  - **FindObject** (UGV, with pic).
  - **Loiter** (UAV, with video).
  - **PTZCollab** (with pic).
  - **LandOnHighestSpot** (UAV, with pic).
  - **EmergencyLanding** (UAV).
  - **ReturnToBase** (UGV).

Figure 8: Corinna (UAV) and DottorCane (UGV).

Use case: Leonardo Drone Contest

BT design choices of LDC25

- Deploy a **BT on each agent**, both sharing a **similar structure**.
- Code **tasks** into a **reusable custom nodes library**.
- Wrap a **task scheduling subtree** inside an agent-specific BT which handles safe initialization and shutdown.

Use case: Leonardo Drone Contest

BT diagram of LDC25

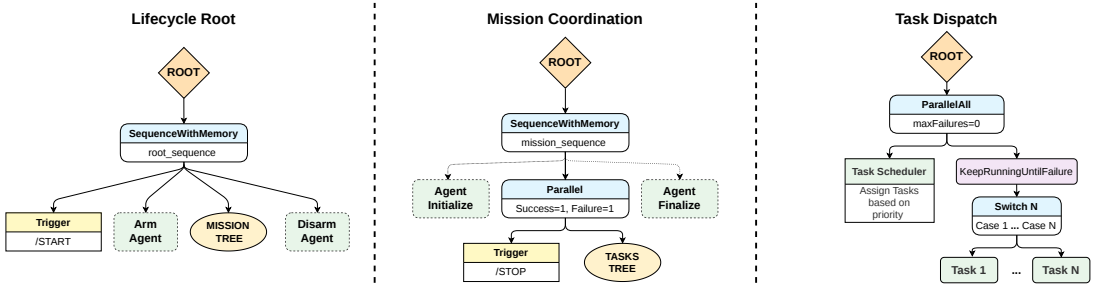


Figure 9: High-level view of the agent-agnostic BT structure: the Lifecycle Root level waits for mission start, arms, then finally disarms the robot; the Mission Coordination level initializes the agent (*e.g.*, takes off), performs the mission watching for a stop command, and finalizes the agent (*e.g.*, lands safely); the Task Dispatch level runs until halted, selecting and executing tasks issued by the Ground Control Station.

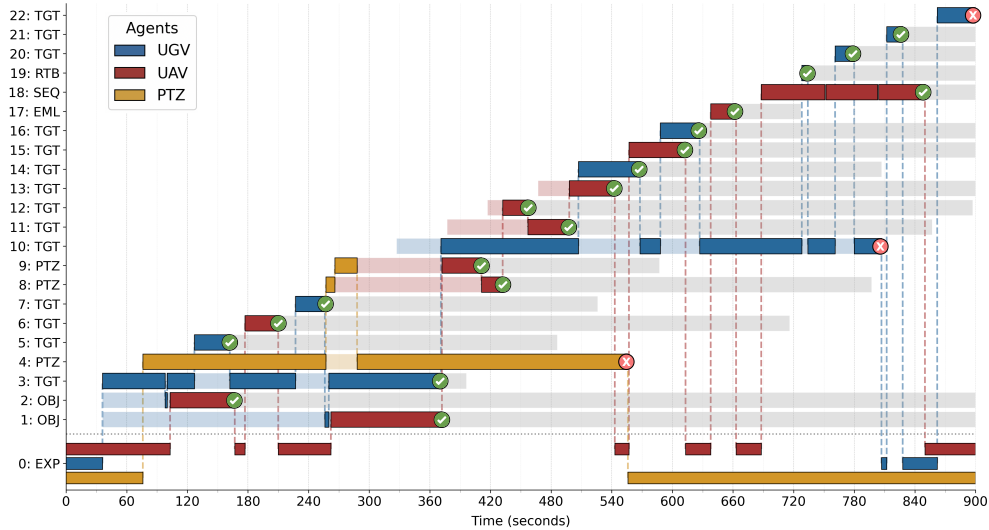
Use case: Leonardo Drone Contest

Strategy of LDC25

- At each tick, **Task Scheduler selects the best task node** which is then ticked.
- If the scheduling selection changes, **preemption** is achieved by **halting** the running node.
- The **scheduling policy** is based on three **tiers**.
  1. **Emergency tasks:** non-preemptable `SyncActionNodes`, always selected first.
  2. **Tasks with non-preemptable portions:** `StatefulActionNodes` that may retain control (*i.e.*, `onRunning()` does not return until a critical operation is completed).
  3. **All other tasks**: evaluated by a cost function with a hysteresis margin.
- Tier 3 cost function evaluates **distance** and an **educated guess of the score**.
- If no valid task is available, an **idle task** of **persistent exploration** is executed.
- **Agents share information** on detected targets: each Task Scheduler keeps a **database of found targets**, used to **navigate directly to known targets** when relevant tasks are issued.

Use case: Leonardo Drone Contest

Task scheduling and execution performance of LDC25



Use case: Leonardo Drone Contest

Task scheduling and execution performance of LDC25

- Over a 20 minutes manche, 23 tasks were issued and 20 were completed successfully, yielding a **completion rate of 87%**.
- Thanks to the **idle exploration task**, 13 out of 19 valid manche targets were **found in the first** $60\,\mathrm{s}$, requiring **10 times less time**.
- **Emergency tasks** were always **scheduled immediately** and completed successfully.

Roadmap

1 Finite-state machines

2 Behavior trees

3 Future directions

Beyond programmed missions

Both **Finite-State Machines** and **Behavior Trees** require the mission to be **programmed in advance**: a state diagram, a tree XML, a set of routines or leaves. The robot can choose, react, recover — but **only along paths a human has spelled out**.

Modern applications increasingly demand:

- **natural-language** or **multi-modal** mission specifications;
- **open-world** requests, not enumerable in advance;
- **on-the-fly assembly of plans** from a library of skills.

Large Language Models (LLMs) are starting to fill this gap, by sitting **on top of** the orchestration layer built so far.

The canonical example is SayCan (Ahn et al., 2022).

- the LLM enumerates candidate **next actions** in natural language, given the **high-level goal** and the **current context**;
- a separate **affordance function**, trained on the robot's actual skills, scores each candidate by **feasibility in the current state**;
- the highest combined-score action is **dispatched to a low-level skill**.

The **LLM** owns **semantics**; the **grounded executor** owns **feasibility**.

The next step is to give the LLM not a free natural-language channel, but a **constrained interface** of typed operations.

Code as Policies (Liang et al., 2022) has the **LLM emit Python code** calling a fixed set of perception and control primitives (`detect_object`, `move_to`, `pick`, ...).
The robot exposes an API, the LLM writes against it.

This is now the industry standard for LLM tool use, formalized by the Model Context Protocol (MCP): a robot can run an **MCP server** that **publishes its capabilities** to any compliant agent, and **receives structured calls** in return.

An alternative to having the LLM **execute** actions one at a time is to have it **author or modify a behavior tree**, which a **deterministic executor** then runs.

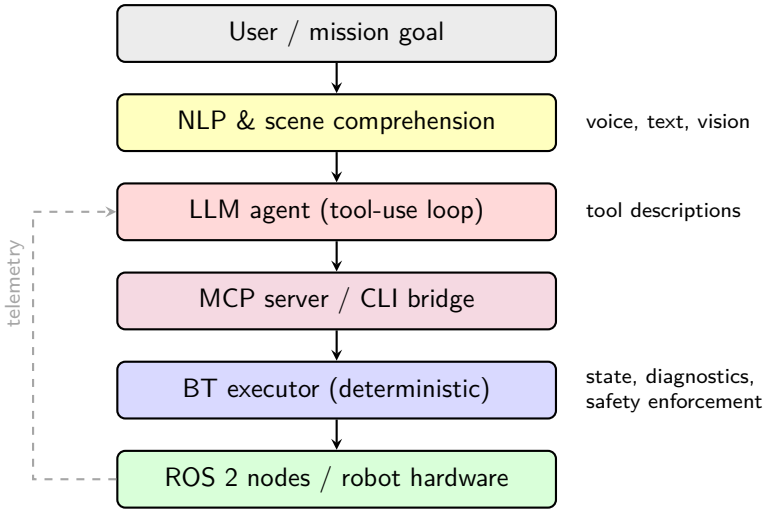
Recent lines of work explore three flavors:

- the LLM **generates** a tree from a natural-language goal, validated and possibly repaired through **feedback loops**;
- the LLM **parameterizes** a hand-crafted tree (chooses sub-goals, populates the blackboard);
- the LLM **selects** subtrees from a library of pre-validated behaviors, via function calling.

The LLM acts as a **planner**, the BT remains the **run-time supervisor**.

An integrated architecture

The stack



An integrated architecture

Layers

A reasonable architecture for an **agent-driven autonomous robot** integrates everything seen so far, in layers (or *harnesses*):

- **input layer**: **NLP** and **scene-understanding** modules turn raw user input (text, voice, images) into a structured high-level request;
- **agent layer**: an **LLM** in a tool-use loop, decomposes the request into calls;
- **interface layer**: **MCP server** (or a CLI bridge) exposing robot capabilities to the agent;
- **orchestration layer**: a **deterministic BT executor** consuming the agent's structured calls;
- **execution layer**: the **ROS 2 nodes** that implement algorithms and drive the hardware.

Each layer's output feeds the next; **telemetry and state** flow back up.

Boundaries and open problems

Why the LLM never gets direct control

Large language models are powerful semantic engines, but they are **not suited to drive a robot directly**.

- **Hallucination**: they can confidently invent capabilities, parameters, or world facts that do not exist.
- **Latency and jitter**: inference times are large and variable, incompatible with control loops.
- **No formal guarantees**: they cannot enforce timing, safety, or constraint contracts.
- **No state ownership**: they have no built-in representation of the robot's diagnostics, mode, or safety envelope.

The deterministic layer below the agent is where the guarantees live.

Boundaries and open problems

Active research questions

The integration of LLM agents into robotic stacks is an active research front.
Open questions include the following.

- **Grounding**: making sure the agent's plans refer to entities that actually exist in the robot's world.
- **Evaluation**: how do you test a system whose behavior is partly emergent?
- **Safety**: where do hard constraints live — in the BT, in a separate runtime monitor, or in a dedicated safety layer?
- **Real-time**: how to keep agent latency from leaking into time-critical loops.
- **Cost and locality**: when can the agent run on-board vs in the cloud, with what trade-offs?

Closing thoughts

Behavior Trees did not replace Finite-State Machines: they coexist, each fitting a different point in the complexity vs reactivity plane.

LLM agents are not poised to replace BTs either: they extend the stack upward, taking over the parts of the problem that humans used to do by hand — understanding what the user wants, and assembling a plan to satisfy it.

The right perspective

The formalisms learned in this lecture are not made obsolete by agentic AI.
They are **moved one level down**, becoming the deterministic substrate that makes agent-driven autonomy **trustable**.